

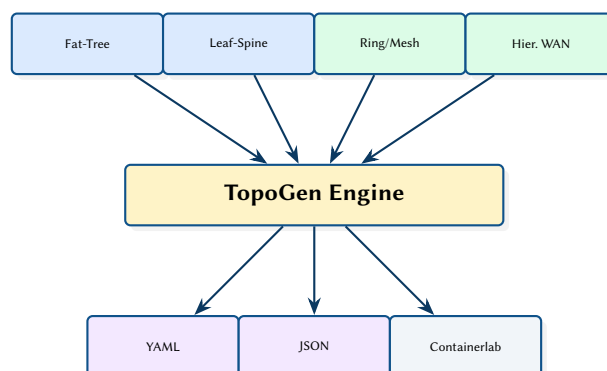
TopoGen: A Seed-Deterministic Network Topology Generator

Datacenter Fabrics, WANs, and Random Graph Models with Geographic Placement

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. TopoGen is a Rust library and CLI for generating realistic network topologies spanning datacenter fabrics (Fat-Tree, Leaf-Spine), wide-area networks (Ring, Mesh, Hierarchical WAN, POP), and random graph models (Barabási–Albert, Watts–Strogatz, Erdős–Rényi). All generation is seed-deterministic via ChaCha8Rng, producing reproducible topologies with realistic interface specifications, vendor-specific naming (Cisco, Arista, Juniper), geographic city placement with distance-aware latency, and gravity-model traffic matrices. Python bindings via PyO3 expose the full API as an installable package.

Contents

1	Introduction and Motivation	2
2	System Architecture	3
3	Datacenter Generators	5
3.1	Fat-Tree	5
3.2	Leaf-Spine	5
4	WAN Generators	6
4.1	Geographic Placement	6
4.2	Hierarchical WAN	7
4.3	POP Generator	7
5	Random Graph Models	7
5.1	Traffic Matrix Generation	8
6	Export Formats	9
7	Design Summary	10
	List of Figures	11
	List of Tables	11
	List of Design Decisions & Rules	12
	Revision History	12
IXP	Internet Exchange Point	7
RNG	Random Number Generator	3

1 Introduction and Motivation

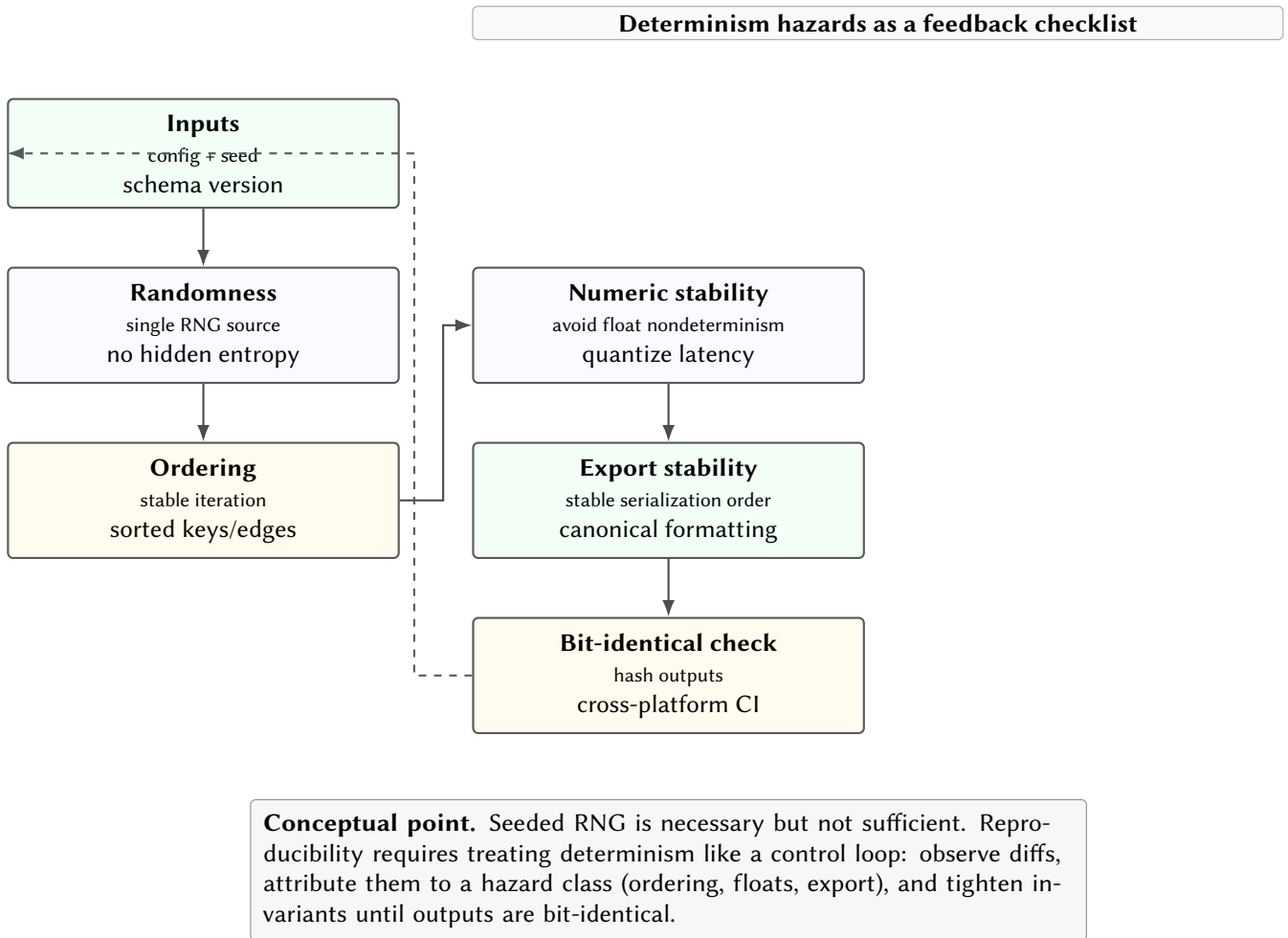


Figure 1. A practical determinism checklist. The fastest way to lose reproducibility is to assume seed is the only input; TopoGen treats ordering, numeric representation, and export canonicalization as first-class hazards.

Network simulation and emulation tools require realistic topologies as input. Existing generators either produce unrealistic random graphs or require manual specification of every node and link. TopoGen bridges this gap by providing parameterised generators for common network architectures, each producing topologies with realistic interface naming, geographic placement, and latency characteristics.

: Seed-Deterministic Generation

Every generator accepts an explicit seed. All randomness flows through ChaCha8Rng seeded from this value, ensuring bit-identical output across platforms and Rust versions.

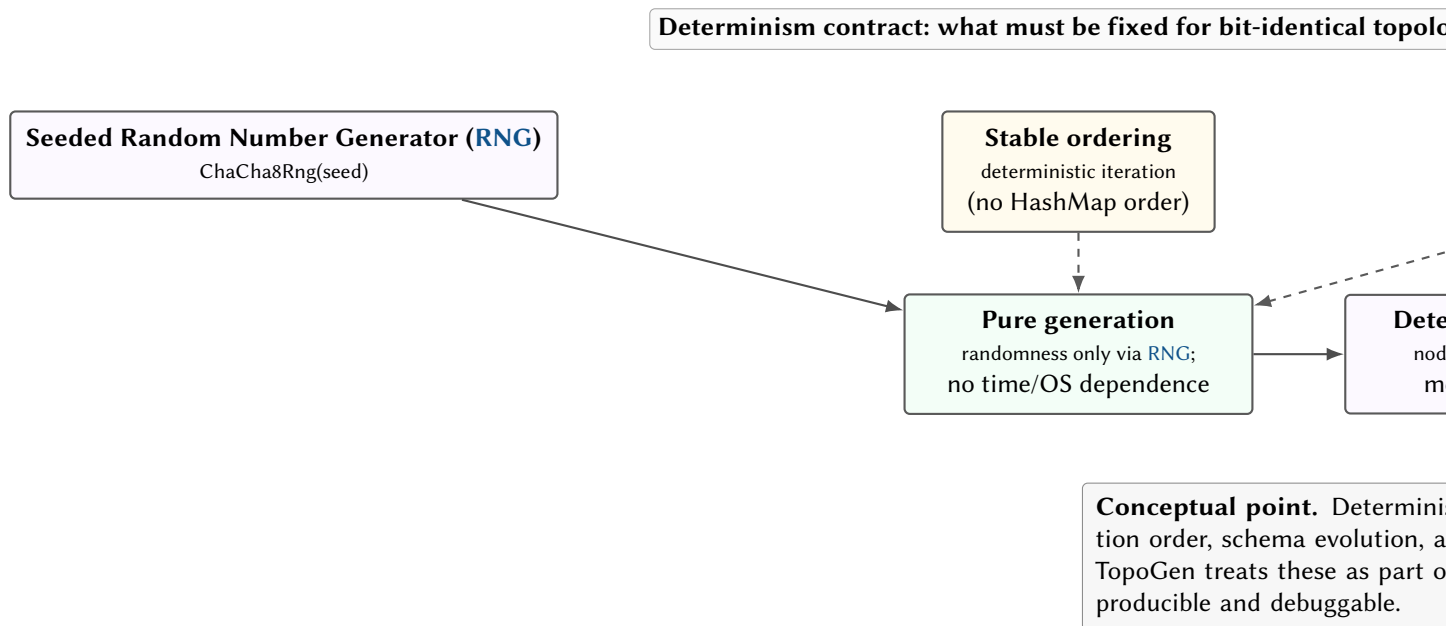


Figure 2. Seed determinism as a system property. A fixed seed is necessary but not sufficient: stable ordering and versioned semantics prevent accidental nondeterminism.

2 System Architecture

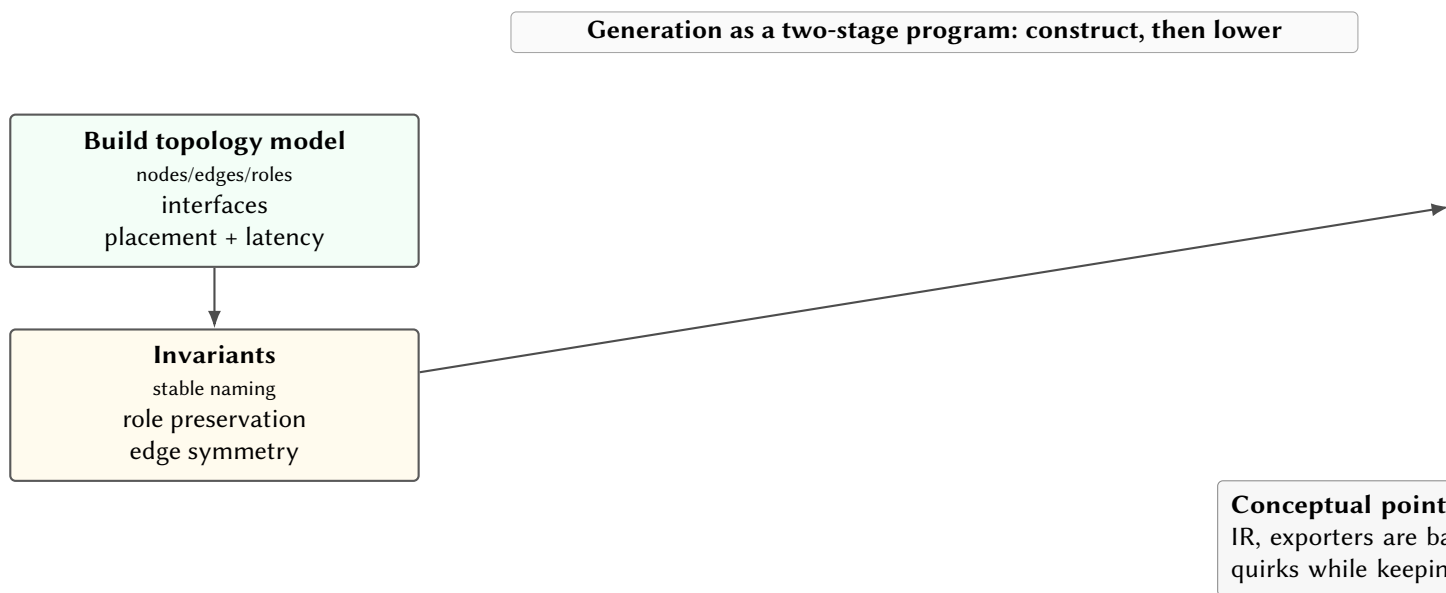


Figure 3. TopoGen's conceptual architecture: build a typed internal model, enforce invariants, then compile to target-specific exports.

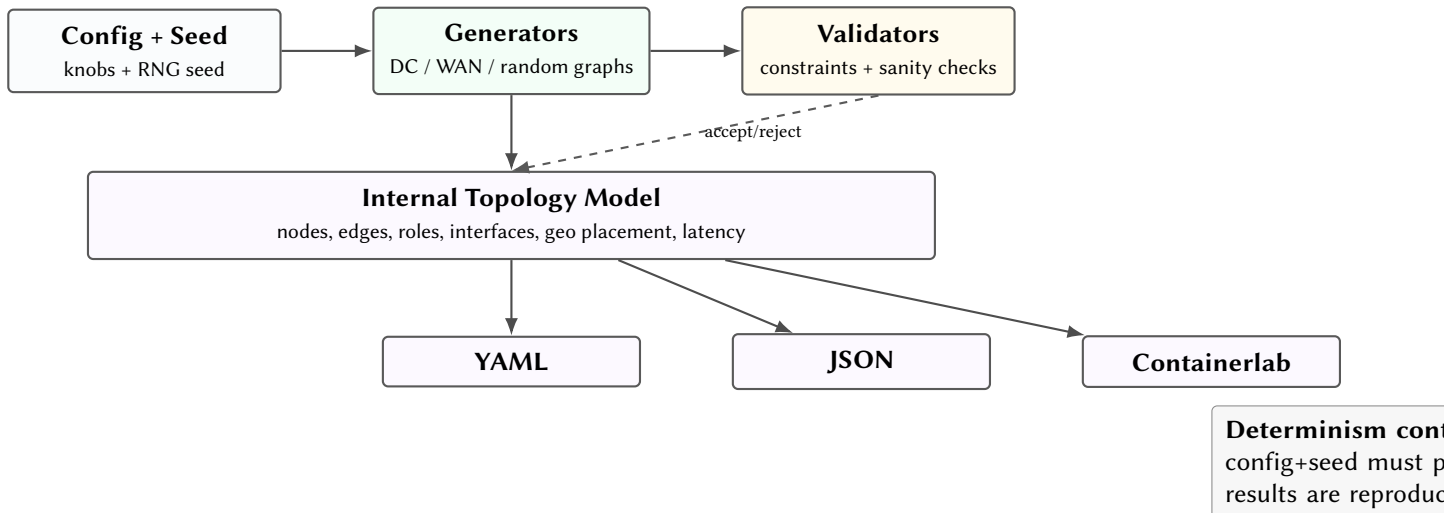


Figure 4. TopoGen end-to-end architecture: seeded generation into a typed internal model, followed by validation and export.

TopoGen is structured as a Rust library crate with a CLI binary and Python bindings:

Table 1. Module organisation.

Module	Language	Role
topogen (lib)	Rust	Core generators, topology model, export
topogen (bin)	Rust	CLI (generate, validate, export)
topogen-py	PyO3	Python package wrapping all generators

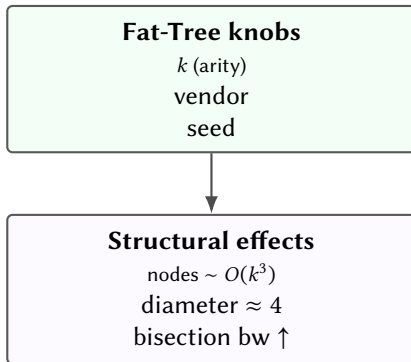
The core topology model stores nodes and edges with typed attributes:

```

pub struct Topology {
    name: String,
    nodes: Vec<Node>,
    edges: Vec<Edge>,
    metadata: TopologyMetadata,
}

pub struct Node {
    pub name: String,
    pub role: NodeRole,           // Core, Spine, Leaf, Access, etc.
    pub vendor: Vendor,          // Cisco, Arista, Juniper
    pub location: Option<GeoLocation>,
    pub interfaces: Vec<Interface>,
}
  
```

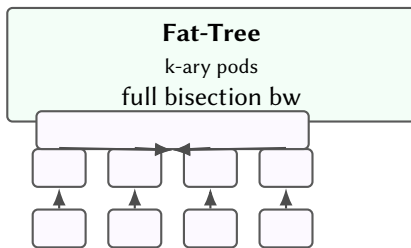
3 Datacenter Generators



Conceptual point. F diversity, bottleneck c effects helps choose th

Figure 5. Datacenter generator knobs and their first-order structural consequences.

Canonical fabric patterns



Why include bottl
uniform traffic, wh
TopoGen exposes b
from topology bias.

Figure 6. Reference diagrams for datacenter fabric generators.

3.1 Fat-Tree

The Fat-Tree generator [1] produces a k -ary topology with three tiers: $k^2/4$ core switches, k pods each containing $k/2$ aggregation and $k/2$ edge switches.

```
pub fn generate_fat_tree(k: usize, seed: u64, vendor: Vendor) -> Topology {
    let mut rng = ChaCha8Rng::seed_from_u64(seed);
    let cores = k * k / 4;
    let pods = k;
    let agg_per_pod = k / 2;
    let edge_per_pod = k / 2;
    // ...
}
```

For $k = 4$, this produces 20 nodes and 32 edges with a network diameter of 4. Interface names follow vendor conventions (e.g. GigabitEthernet0/0 for Cisco, Ethernet1 for Arista).

3.2 Leaf-Spine

The Leaf-Spine generator creates a two-tier fabric with configurable spine and leaf counts. Every leaf connects to every spine, producing $S \times L$ links for S spines and L leaves.

Design Decision 1: Vendor-Specific Interface Naming

Interface names are generated by a `VendorNamer` trait with implementations for Cisco (GigabitEthernet, TenGigE), Arista (Ethernet), and Juniper (et-/xe-/ge-). Port numbering is deterministic from the `RNG` state, producing realistic but reproducible configurations.

4 WAN Generators

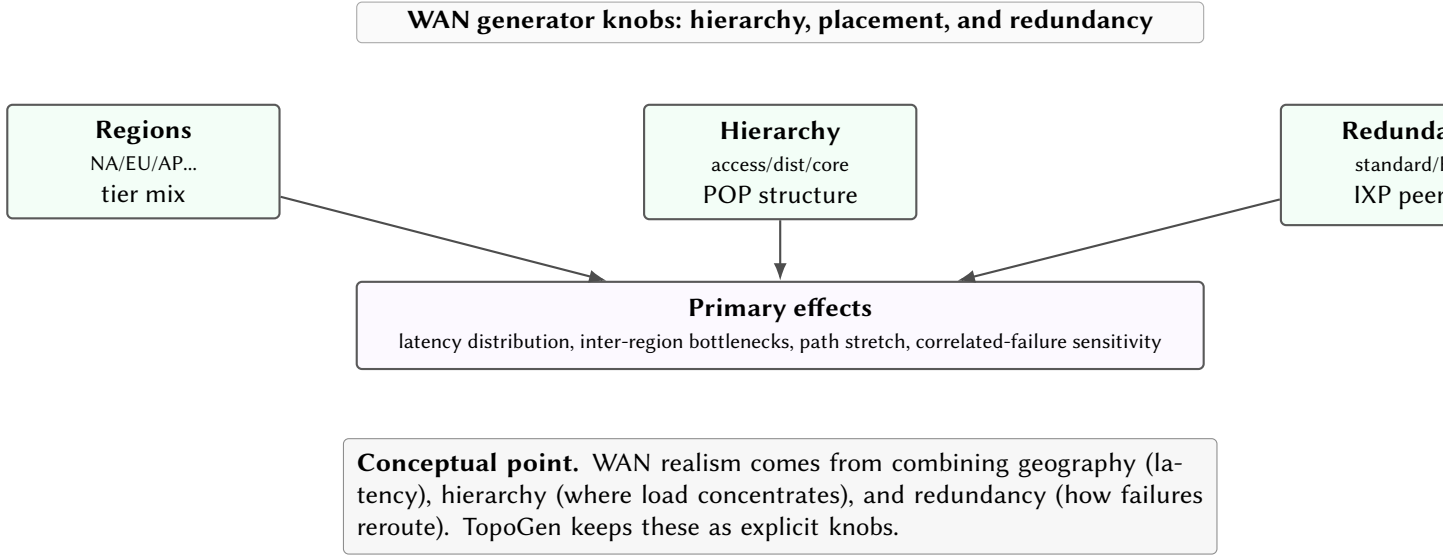


Figure 7. WAN generators expose knobs that directly shape latency and failure behaviour, not just node counts.

4.1 Geographic Placement

WAN generators place nodes at real-world city locations. Cities are organised by region (NA, EU, AP, etc.) and tier (1 = major hubs, 2 = secondary, 3 = regional). The generator selects cities from the requested region and tier, then computes inter-node latency from great-circle distance:

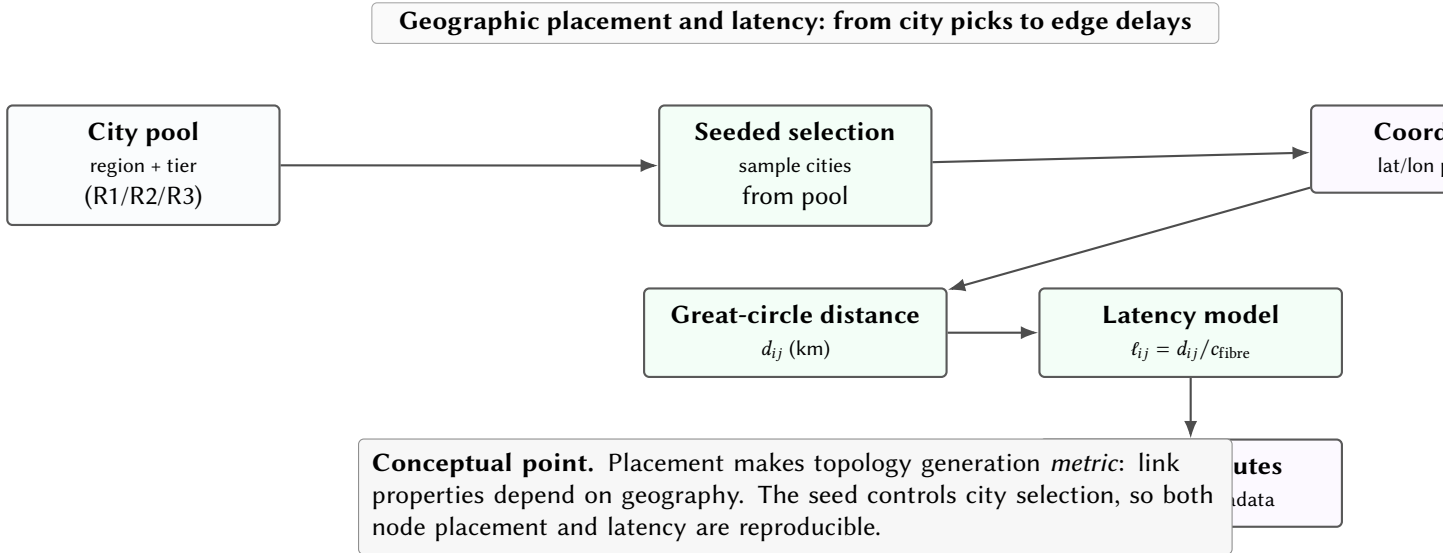


Figure 8. Geographic placement as a reproducible pipeline: pick cities, assign coordinates, compute distances, then derive per-edge latency.

$$\text{latency}(\mu\text{s}) = \frac{d_{\text{km}} \times 1000}{c_{\text{fibre}}} \quad \text{where} \quad c_{\text{fibre}} \approx 200,000 \text{ km/s} \quad (1)$$

4.2 Hierarchical WAN

The hierarchical generator produces a three-tier access/distribution/core topology with multi-region support and configurable redundancy levels (standard, high). Each region gets a proportional allocation of nodes based on a gravity model.

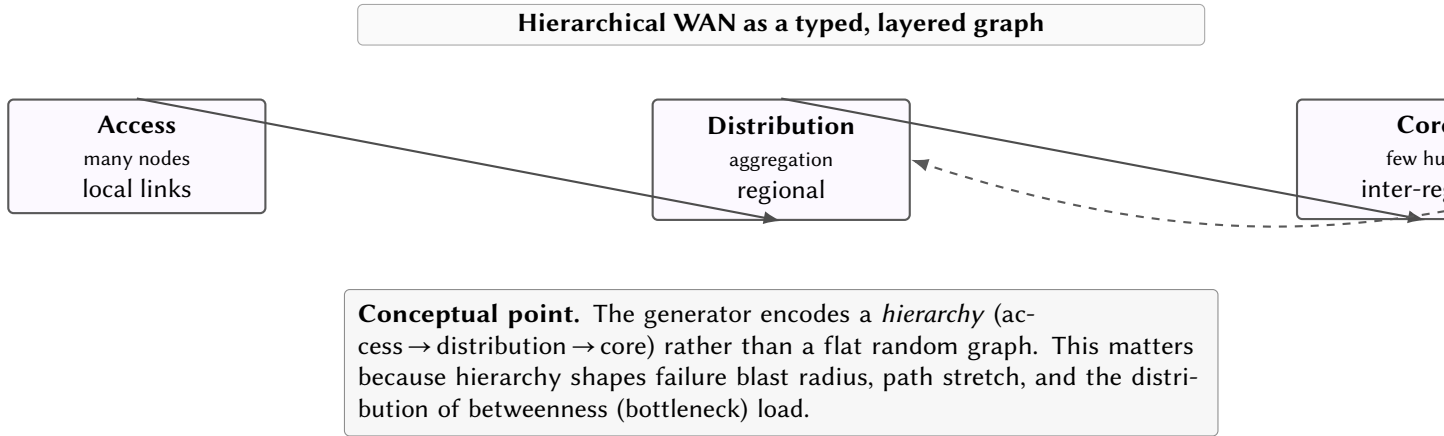


Figure 9. Hierarchical WAN structure. Redundancy increases path diversity by adding parallel connectivity at distribution/core tiers.

4.3 POP Generator

The POP generator models ISP-style backbones with internal POP structure. Each POP contains border routers, route reflectors, and access routers. Optional Internet Exchange Point (IXP) peering nodes create inter-POP connectivity.

5 Random Graph Models

Three classic random graph models are available:

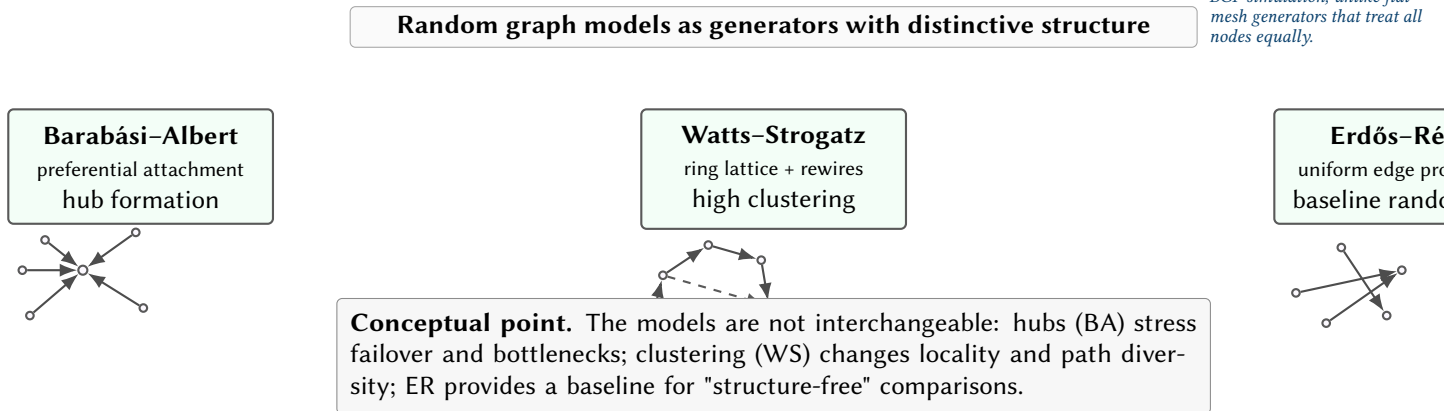


Figure 10. Random graph models encode different structural priors. TopoGen exposes them as controlled alternatives for sensitivity studies.

Table 2. Random graph model parameters.

Model	Parameters	Property
Barabási–Albert [2]	n, m	Scale-free degree distribution
Watts–Strogatz [3]	n, k, β	Small-world clustering
Erdős–Rényi	n, p	Uniform random edges

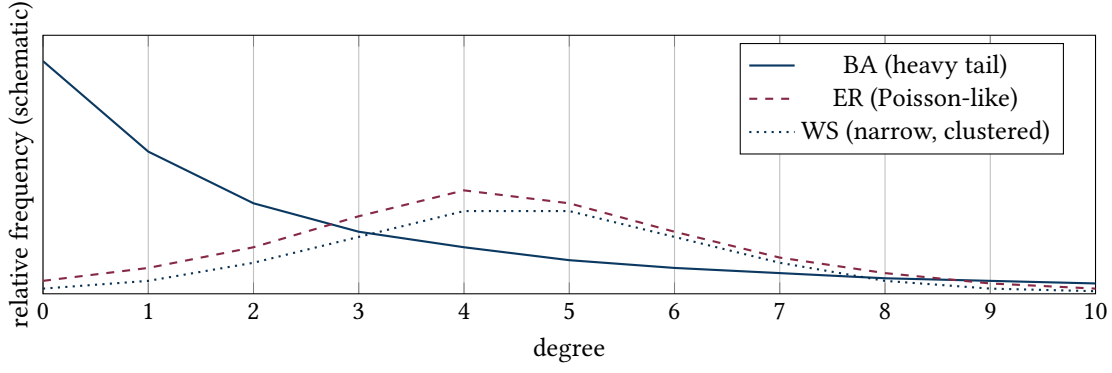


Figure 11. Schematic degree distribution shapes for common random graph models. The qualitative differences (heavy-tail vs narrow vs Poisson-like) drive bottlenecks and failure sensitivity.

5.1 Traffic Matrix Generation

TopoGen can generate gravity-model traffic matrices for any topology. The traffic demand between nodes i and j is proportional to the product of their “mass” (degree or configured weight):

$$T_{ij} = \alpha \cdot \frac{m_i \cdot m_j}{d_{ij}^\gamma} \quad (2)$$

where d_{ij} is the shortest-path distance, γ controls distance decay, and α is a normalisation constant.

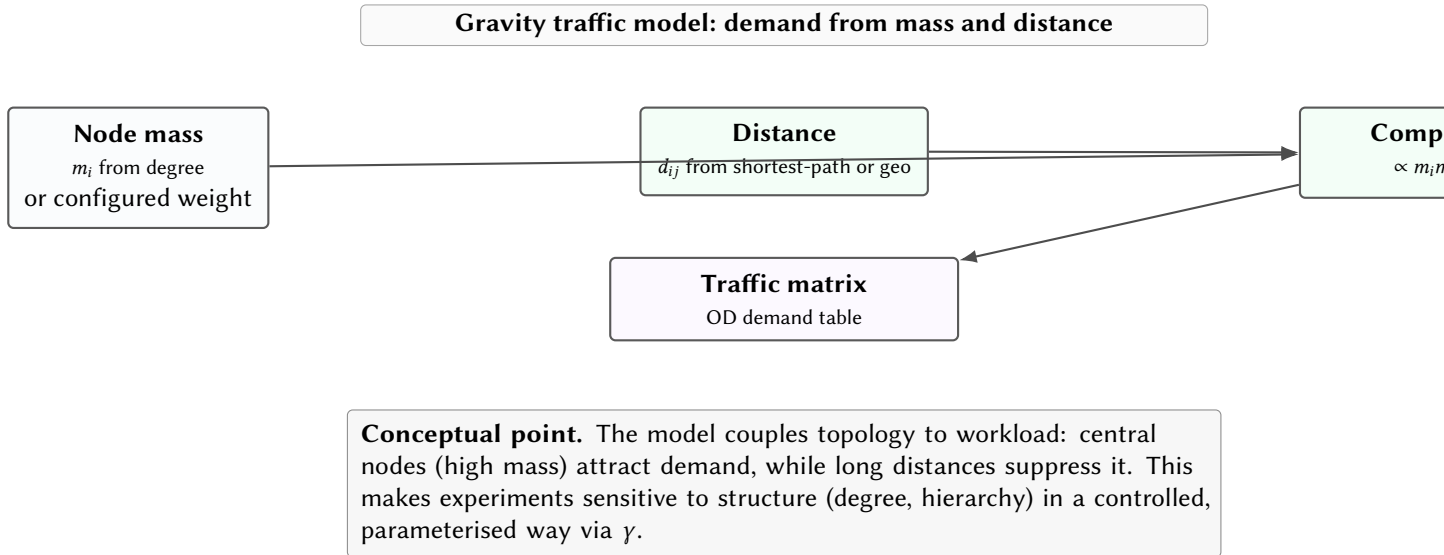


Figure 12. Gravity-model demand generation. A tunable distance exponent γ controls how strongly long paths are penalised.

6 Export Formats

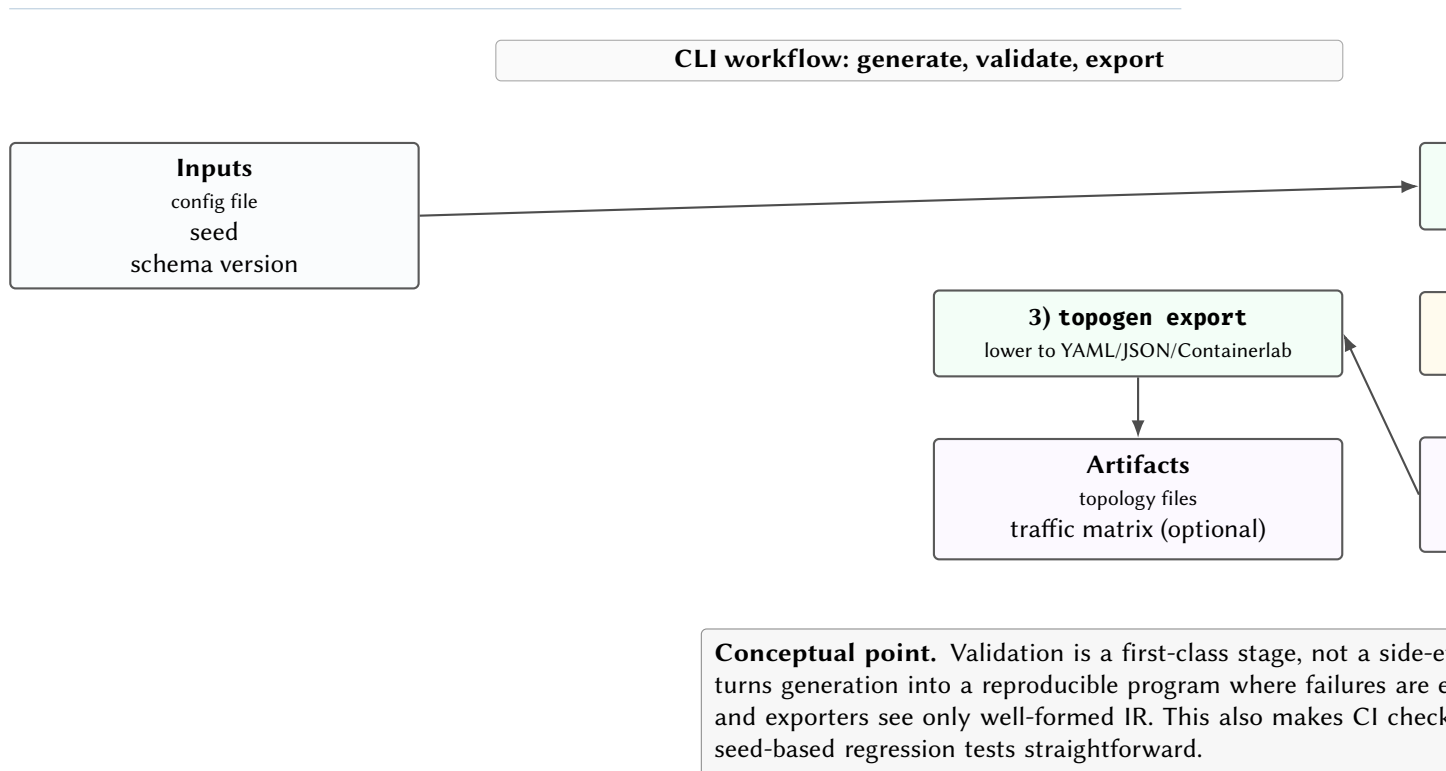


Figure 13. TopoGen’s CLI as a pipeline. The commands correspond to explicit stages in the generator architecture: construct the model, validate invariants, then lower to target-specific outputs.

Topologies can be exported to YAML, JSON, and Containerlab (`.clab.yaml`) formats. The Containerlab exporter maps node roles to container images and generates the topology file ready for `clab deploy`.

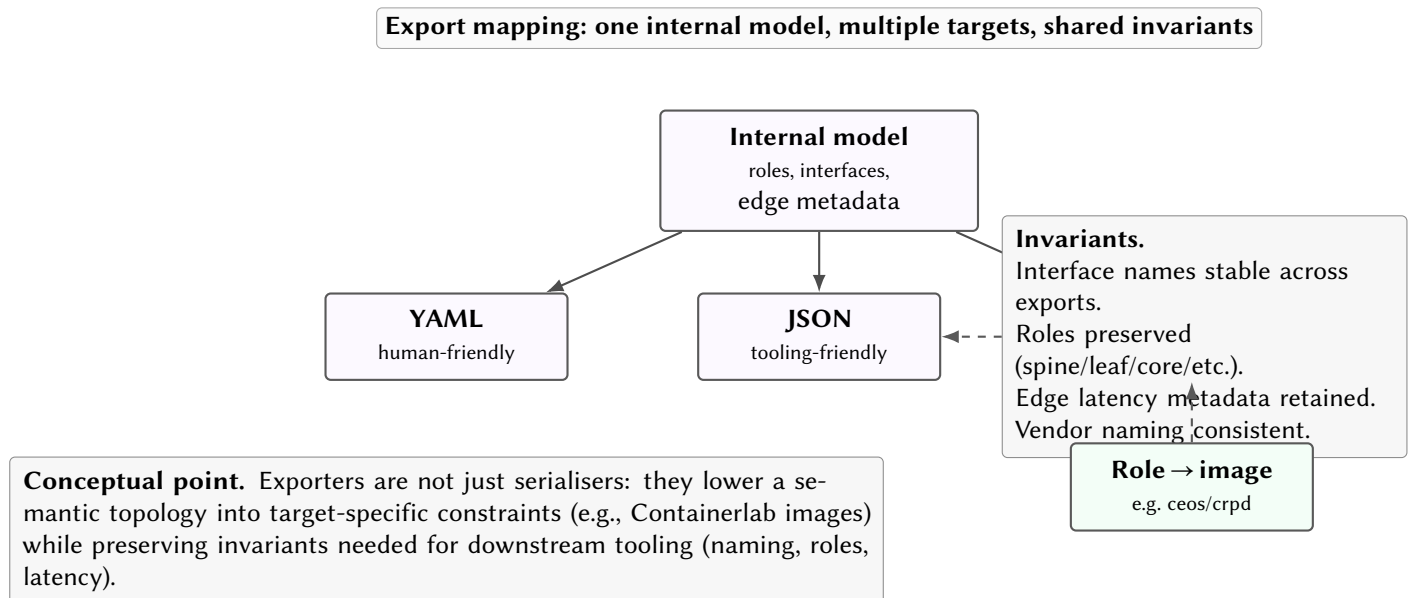


Figure 14. Export as lowering. All outputs originate from the same internal model; target-specific mappings are explicit, and shared invariants keep downstream consumers consistent.

Design Decision 2: Containerlab Integration

The exporter maps vendor and role to Docker images (e.g. `ceos:latest` for Arista, `crpd:latest` for Juniper). Interface names in the Containerlab output match those in the generated topology, enabling direct use of TopoGen configs in network emulation labs.

7 Design Summary

Table 3. Key design decisions.

Decision	Rationale
ChaCha8Rng	Cross-platform determinism; StdRng algorithm is unspecified
Vendor-specific naming	Realistic configs for lab deployment and parser testing
Geographic city DB	Distance-aware latency without external dependencies
Gravity traffic model	Produces realistic demand patterns from topology structure alone
PyO3 bindings	Native Python package for integration with simulation frameworks

References

- [1] M. Al-Fares, A. Loukissas, and A. Vahdat, “A scalable, commodity data center network architecture,” in *SIGCOMM*, 2008.
- [2] A.-L. Barabási and R. Albert, “Emergence of scaling in random networks,” *Science*, vol. 286, pp. 509–512, 1999.
- [3] D. J. Watts and S. H. Strogatz, “Collective dynamics of ‘small-world’ networks,” *Nature*, vol. 393, pp. 440–442, 1998.

List of Figures

1	A practical determinism checklist. The fastest way to lose reproducibility is to assume seed is the only input; TopoGen treats ordering, numeric representation, and export canonicalization as first-class hazards.	2
2	Seed determinism as a system property. A fixed seed is necessary but not sufficient: stable ordering and versioned semantics prevent accidental nondeterminism.	3
3	TopoGen’s conceptual architecture: build a typed internal model, enforce invariants, then compile to target-specific exports.	3
4	TopoGen end-to-end architecture: seeded generation into a typed internal model, followed by validation and export.	4
5	Datacenter generator knobs and their first-order structural consequences.	5
6	Reference diagrams for datacenter fabric generators.	5
7	WAN generators expose knobs that directly shape latency and failure behaviour, not just node counts.	6
8	Geographic placement as a reproducible pipeline: pick cities, assign coordinates, compute distances, then derive per-edge latency.	6
9	Hierarchical WAN structure. Redundancy increases path diversity by adding parallel connectivity at distribution/core tiers.	7
10	Random graph models encode different structural priors. TopoGen exposes them as controlled alternatives for sensitivity studies.	7
11	Schematic degree distribution shapes for common random graph models. The qualitative differences (heavy-tail vs narrow vs Poisson-like) drive bottlenecks and failure sensitivity.	8
12	Gravity-model demand generation. A tunable distance exponent γ controls how strongly long paths are penalised.	8
13	TopoGen’s CLI as a pipeline. The commands correspond to explicit stages in the generator architecture: construct the model, validate invariants, then lower to target-specific outputs.	9
14	Export as lowering. All outputs originate from the same internal model; target-specific mappings are explicit, and shared invariants keep downstream consumers consistent.	9

List of Tables

1	Module organisation.	4
2	Random graph model parameters.	7
3	Key design decisions.	10

List of Design Decisions & Rules

1	Design Rule: Seed-Deterministic Generation	2
1	Vendor-Specific Interface Naming	6
2	Containerlab Integration	9

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report